UMBONO CLOUD SERVICES × AWS × FIRST DISTRIBUTION

Community Safety Command Centre

Build an AI-powered incident reporting and triage platform for safer South African communities

Monday 27 October 2026 • One-Day Hackathon

The Problem

South African communities face rising safety challenges. Residents report incidents through fragmented channels — WhatsApp groups, phone calls, social media — with no centralised view, no prioritisation, and slow response times. Critical incidents get lost in the noise.

Your Mission

Build a **Community Safety Command Centre** — a web application where residents can report safety incidents and AI automatically triages, categorises, and prioritises them so response teams can act on what matters first.

> **You are NOT building from scratch.** We've provisioned your AWS environment with a working skeleton — DynamoDB tables (pre-seeded with sample data), Lambda functions with guided TODO markers, an API Gateway, and an S3 bucket. Your job is to complete the TODOs and build a compelling frontend.

Architecture



The Three TODOs

Open your Lambda functions in the AWS Console. You'll find these TODO markers:

TODO #1

AI Triage with Amazon Bedrock

Call Bedrock (`anthropic.claude-haiku-5-5`) to analyse each incident report. The AI should assess severity (critical / high / medium / low), generate a one-line summary, and suggest a category.

Lambda: `team{N}-submit-report`

TODO #2

Improve DynamoDB Storage

The save-to-DynamoDB code works, but it's basic. Add input validation, geolocation coordinates, photo URL support, or any other fields that would make the data richer and more useful.

Lambda: `team{N}-submit-report`

TODO #3

Improve the Query

The current `get-reports` does a full table scan — expensive at scale. Add a Global Secondary Index, pagination, sorting by severity, location-based filtering, or full-text search.

Lambda: `team{N}-get-reports`

Beyond the TODOs

The TODOs are the minimum. To win, go further:

- **React frontend** — a compelling UI with a report form, incident dashboard, and map view
- **Photo uploads** — use your S3 bucket for incident photos
- **Real-time updates** — polling, WebSocket, or SSE for live incident feeds
- **Status workflow** — new → acknowledged → dispatched → resolved
- **Analytics** — incident heatmaps, response time metrics, category breakdowns
- **Multi-language** — support for isiZulu, Afrikaans, or other SA languages

API Reference

Submit an incident:

```
POST /reports
Content-Type: application/json

{
  "reporterName": "Jane Smith",
  "incidentType": "theft",
```

```
  "location": "Corner of Rivonia Rd and 5th Street",
  "description": "Witnessed a phone snatching. Suspect fled on foot."
}
```

Response:

```
{
  "message": "Incident reported successfully",
  "incidentId": "a1b2c3d4-...",
  "severity": "medium",
  "aiSummary": "Phone snatching on Rivonia Rd - suspect fled north"
}
```

Get all incidents:

```
GET /reports
GET /reports?status=new
GET /reports?type=theft
```

Response:

```
{
  "incidents": [
    {
      "incidentId": "a1b2c3d4-...",
      "timestamp": "2026-10-27T09:30:00Z",
      "reporterName": "Jane Smith",
      "incidentType": "theft",
      "location": "Corner of Rivonia Rd and 5th Street",
      "description": "Witnessed a phone snatching...",
      "severity": "medium",
      "aiSummary": "Phone snatching - suspect fled north",
      "aiCategory": "theft",
      "status": "new"
    }
  ],
  "count": 1
}
```

Valid incidentType values: `suspicious_activity`, `theft`, `vandalism`, `traffic`, `fire`, `medical`, `noise`, `infrastructure`, `other`

Judging Criteria

AI Integration — 25%

Effective use of Bedrock for triage, quality of prompts, accuracy of categorisation

Technical Execution — 25%

Code quality, architecture decisions, error handling, use of AWS services

User Experience — 25%

Frontend design, usability, accessibility, mobile responsiveness

Community Impact — 25%

Would this actually help a community? Is it realistic and deployable?

Schedule

08:30	Registration & breakfast
09:00	**Kickoff** — challenge brief, environment walkthrough, Q&A
09:30	**Hack begins!** Mentors available throughout
12:30	Lunch (keep hacking!)
15:00	**Code freeze** — stop coding, prepare demo
15:15	Team presentations (7 min each + 3 min Q&A)
16:00	Judging deliberation
16:30	**Winners announced** & closing

Quick Reference

Region: `af-south-1` (Cape Town)
Runtime: Python 3.12
Bedrock Model: `anthropic.claude-haiku-5-5`
API: `POST /reports` (submit) • `GET /reports` (list)
DynamoDB Key: `incidentId` (HASH) + `timestamp` (RANGE)
Pre-seeded data: 3 sample incidents per team — query them to see the schema

Rules

- Use only your team's AWS resources — do not access other teams' tables, buckets, or functions
- AI assistants (Claude, ChatGPT, Copilot) are **encouraged** — this is about building, not memorising syntax
- All code must be written during the hackathon — no pre-built projects
- You may use any open-source libraries, frameworks, or starter templates
- Final demo must run live against your AWS environment